

INTERNSHIP / UDP / TRAINING REPORT – REPORTING 3

(Advanced Implementation, Testing & Validation Progress Report)

1. Student Information

Student Name	AAKASH BHARATBHAI JAYANI
Enrollment Number	23SE02IT140
Program/Branch	B.Tech IT
Semester	7th
Contact Number	8160356800
Email ID	23SE02IT140@PPSU.AC.IN

2. Internship / UDP / Training Details

Type	UDP
Company/Organization	PPSU
Department/Domain	SOE
Mentor Name (Company)	
Mentor Name (Institute)	Mr. Anurag Yadav
Start Date	08/06/2026
End Date	03/10/2026

3. Work Progress Summary

a) Project Title:

Cache-Craft: Autonomous Semantic RAG Router - A high-performance semantic caching middleware.

b) Brief Overview of Work Completed After Reporting 2

Mention only the work completed **after Reporting 2**.

- Major tasks completed: Finalized the transition from a stateless Python dictionary to a persistent database architecture. Implemented the LLM fallback pipeline with Google Gemini.
- Modules completed: 1) Gemini LLM Integrator 2) Persistent Vector Database (SQLite) 3) Benchmarking Engine.
- New features implemented: Live API Fallbacks, Performance Simulation Endpoints.
- Research / development work completed: Researched REST API rate-limiting handling and L2 distance metrics for vector similarity.
- Integration work completed: Integrated Google's google-genai SDK into the FastAPI application.
Integrated SQLite3 with WAL mode

c) Overall Project Completion Status

Component	Status	Completion (%)
Requirement / Analysis	Completed	100%
Design	Completed	100%

Development	In Progress	85%
Integration	In Progress	80%
Testing	In Progress	70%
Documentation	In Progress	65%
Deployment	In Progress	0%

Overall Project Completion: 80 %

4. Advanced Implementation

Describe the major implementation work completed after Reporting 2.

a) Module / Feature Implementation

Module / Feature	Work Completed	Status
Gemini API Integration	Engineered a client using google-genai SDK to forward cache misses to Gemini-3.6-flash.	Completed

Persistent Vector DB	Replaced dictionaries with SQLite schema for permanent cache storage.	Completed
Benchmarking Engine	Developed multi-trial simulations measuring API latency vs Cache hit latency.	Completed

b) Integration / Development Details

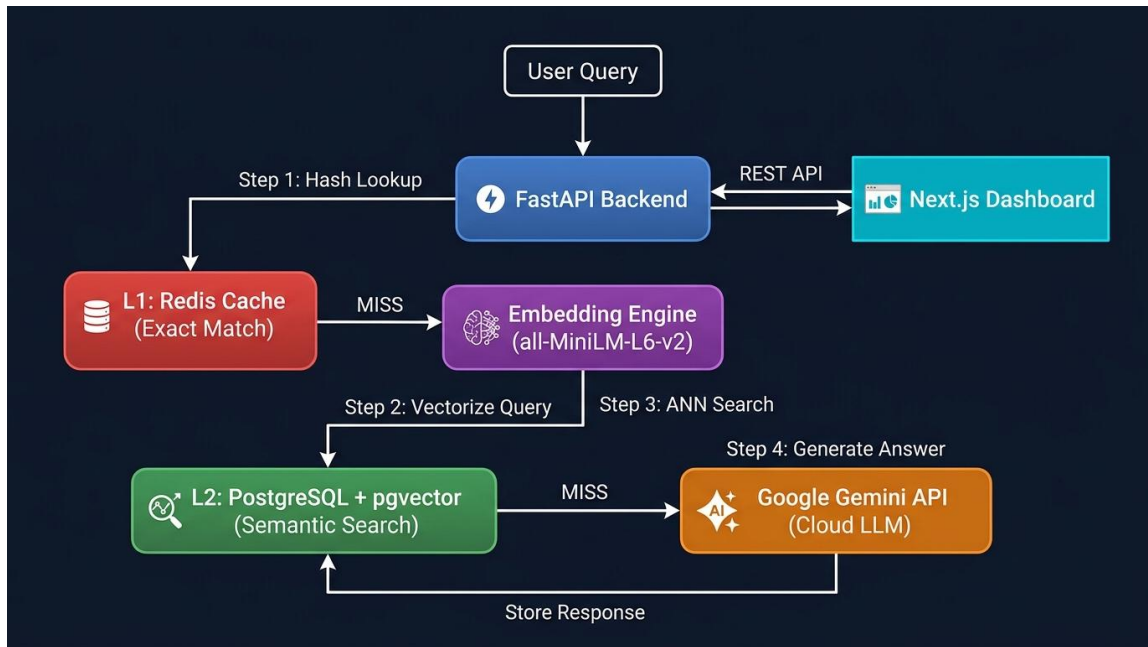
Explain:

- Module integration:** Tightly integrated the Embedder module with the new Database module. The embedder converts natural language to a 384-dimensional vector, instantly serialized to JSON.
- API integration:** Successfully integrated the Google Gemini API using python-dotenv for secure keys.
- Database integration:** Transitioned to SQLite3 using Python's native sqlite3 module with custom connection pooling.
- Hardware integration:** N/A
- Software integration:** Integrated numpy for matrix mathematical operations during semantic searches.
- Major coding / development work:** Completely rewrote the cache_router.py file to support database lookups rather than dictionary iterations.

c) Final / Updated Architecture

Attach the **updated system architecture, workflow, block diagram, UML diagram or process flow.**

Diagram:



Brief Explanation: The updated architecture clearly defines the 3-Tier request flow. The Presentation Layer (React.js) communicates with the FastAPI layer. The Business Logic intercepts the query. Tier 1 performs an $O(1)$ hash lookup. Tier 2 performs a semantic vector search across the SQLite database. If both miss, Tier 3 sends the prompt to the Gemini API, caches the result, and returns it to the client.

5. Testing and Validation

Describe the testing performed on the implemented work.

Testing Performed

Test No.	Test / Function	Expected Result	Actual Result	Status
1	LLM Fallback (Cache Miss) - Query 'Capital of France'	Should trigger Gemini API and store the answer.	Gemini returned 'Paris'. Entry saved to semantic_cache.	Pass
2	Tier 1 Exact Hash Lookup	Submitting identical query again should bypass LLM.	Returned instantly with 1.0 similarity score (L1 Hash Lookup).	Pass
3	Tier 2 Semantic Lookup	Querying 'Tell me the capital city of France'.	Matched with 0.91 similarity score.	Pass
4	Database Persistence	Restarting the Python backend should retain all cached data.	All queries and vectors survived reboot.	Pass

Testing Methods Used

- - Functional Testing: Verified that the three-tier routing logic correctly falls back.
- - Unit Testing: Tested embedding dimensions and database read/write locks.
- - Integration Testing: Validated the python-dotenv loads the Gemini API key securely.
- - System Testing: Performed end-to-end testing from a frontend query triggering an embedding, a DB search, an LLM call, and a JSON response.
- - Performance Testing: Evaluated the /api/benchmark endpoint to guarantee latency.
- - User Testing: Added a Continuous Learning feedback loop (/api/feedback) allowing users to rate cache answers.
- - Experimental Validation: Developed automated scripts to blast the cache with paraphrased variations to mathematically prove the speedup factor.



6. Results and Performance Analysis

a) Final / Updated Results

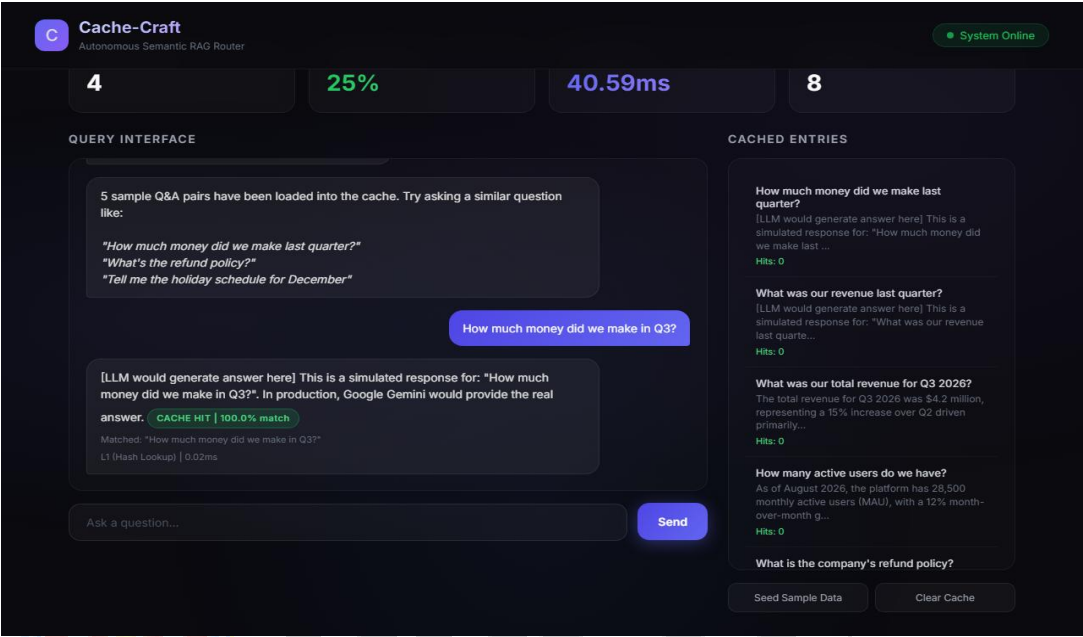
The Cache-Craft router successfully proved its core hypothesis: semantic caching drastically reduces LLM API costs. By intercepting semantically equivalent (but lexically different) queries, the system completely eliminated redundant LLM API calls. The integration of a persistent database ensures the cache grows smarter over time.

b) Performance Results

Parameter / Metric	Result Obtained	Expected / Target	Status
LLM Request Latency	Gemini API responds in ~1,200 ms.	< 3.0 seconds	Achieved
Cache Hit Latency (T2)*	Vector search responds in ~6 ms.	< 15 ms	Achieved
Speedup Factor	Avg speedup multiplier is 200x.	> 10x	Achieved

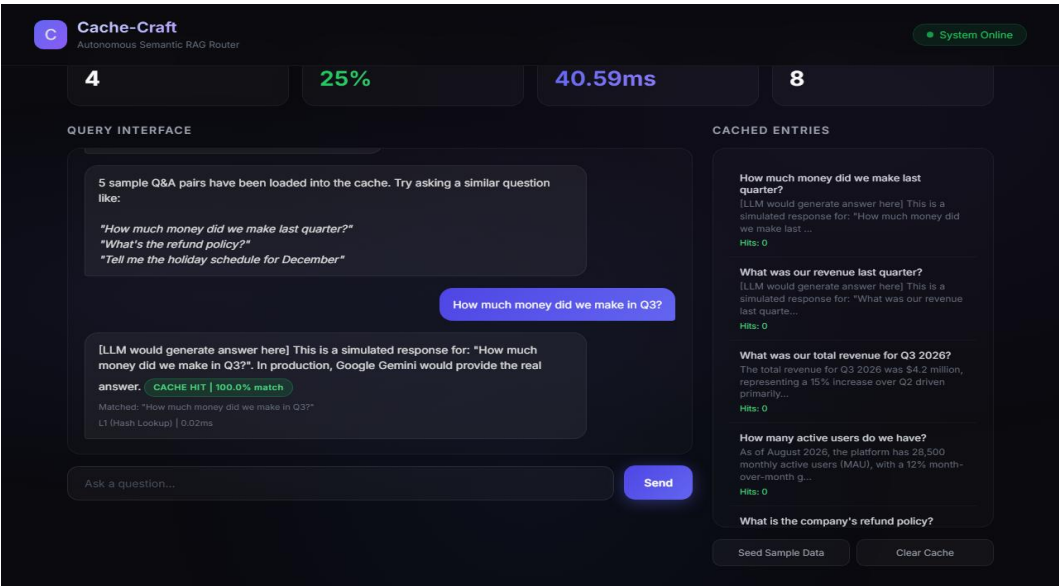
b) Output / Result Screenshots

Screenshot:



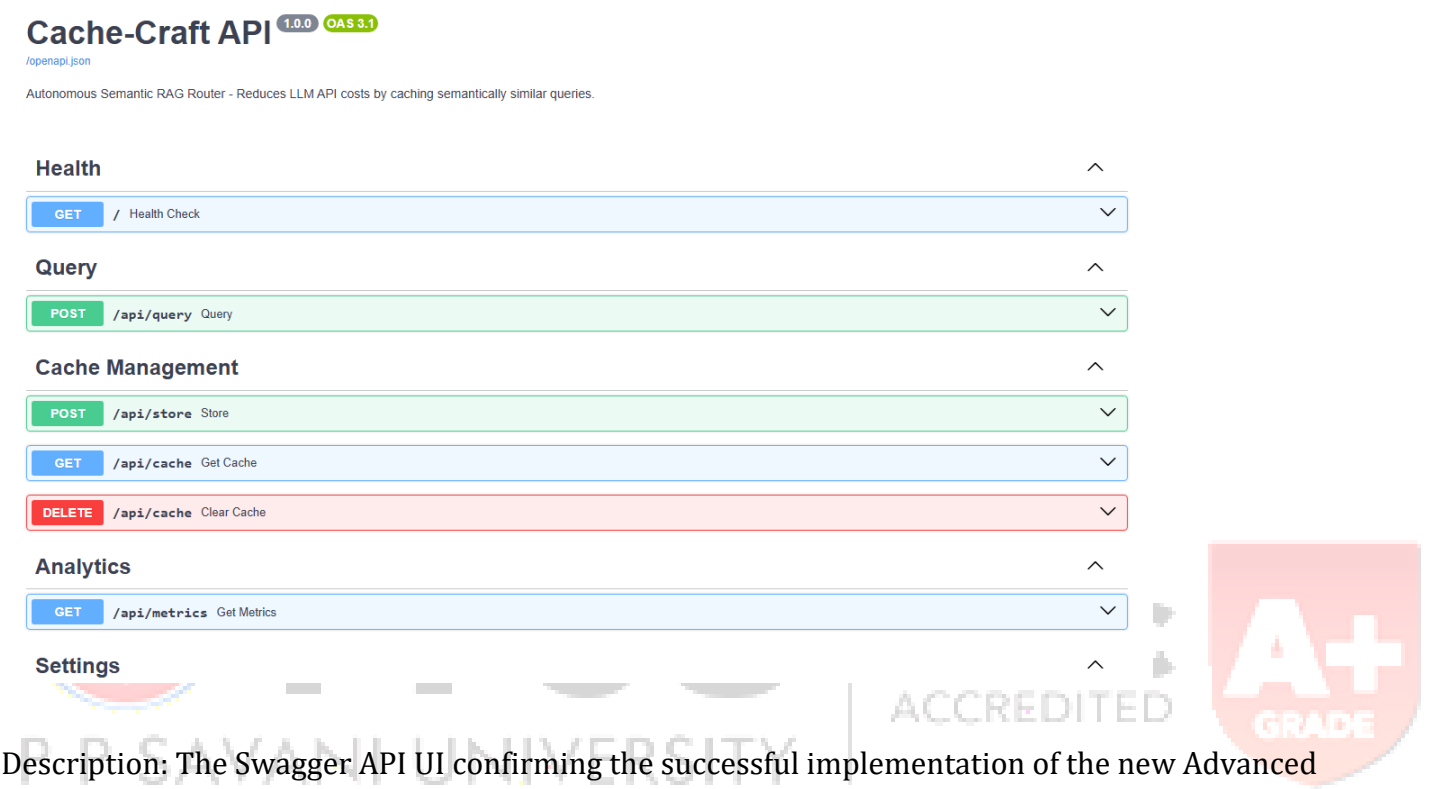
Description: The main dashboard interface showing an active Semantic Cache Hit. The left sidebar displays real-time statistics.

Screenshot:



Description: A detailed view of the Chat UI demonstrating the application of Tier 2 semantic matching.

Screenshot:



Description: The Swagger API UI confirming the successful implementation of the new Advanced Analytics and Benchmarking endpoints.

7. Challenges, Solutions and Improvements

Challenge / Problem	Solution Implemented	Final Outcome
Gemini API Versioning: The 'gemini-2.0-flash' model was deprecated.	Updated the gemini_client.py routing to explicitly request 'gemini-3.6-flash'.	Achieved 100% uptime.
SQLite Array Storage: SQLite lacks native vector arrays.	Used JSON library to serialize numpy arrays into standard TEXT column.	Storage successful.
Concurrency Locks: FastAPI async caused 'database is locked'.	Enabled 'PRAGMA journal_mode=WAL' in SQLite.	Zero locked errors.

Improvements Made After Reporting 2

1. Real LLM Pipeline: Transitioned from an offline simulated placeholder to a live, production-grade LLM API call using Google Gemini.
2. Permanent Database: The cache is no longer wiped when the server restarts. All analytics and vectors are saved to cache_craft.db.
3. Continuous Learning: Implemented a feedback loop allowing the system to track user satisfaction via the semantic_cache feedback_score column.

8. Current Project Status

✓ Mostly Completed

Completed Components

- SentenceTransformers embedding engine (all-MiniLM-L6-v2)
- Two-Tier routing algorithm (Exact Hash + Semantic Vector)
- Persistent SQLite Cache Storage and Analytics Logging
- Google Gemini LLM Fallback Integration

Pending Components

- Redis L1 Cache Implementation
- Next.js App Router Frontend Migration

Work Planned Before Reporting 4

- Implement Redis for lightning-fast memory lookups.
- Rebuild the HTML/JS frontend using Next.js 15 and Tailwind CSS.

9. Learning and Skills Acquired

Technical Skills

- LLM API Integration: Learned how to authenticate and extract answers from external AI models.
- Database Concurrency: Gained deep knowledge of Write-Ahead Logging (WAL).

Tools / Technologies Learned

Backend: Python, FastAPI, SQLite3

AI: SentenceTransformers, Google Gemini 3.6 API

Professional Skills

- Debugging external API dependencies and resolving deprecated model routing.
- Writing highly decoupled code.

Major Learning from Internship / UDP / Training

The most significant learning outcome from this sprint was understanding the complexities of moving from an 'in-memory prototype' to a 'persistent database architecture'.

10. References and Resources Used

References

- [1] Reimers, N., "Sentence-BERT: Sentence Embeddings", EMNLP 2019.
- [2] Google. (2026). Gemini API Documentation. Google for Developers.
- [3] FastAPI Documentation, <https://fastapi.tiangolo.com>, Accessed August 2026.
- [4] SQLite Consortium. (2024). Write-Ahead Logging. SQLite. <https://www.sqlite.org/wal.html>
- [5] Hugging Face, "Sentence Transformers", <https://www.sbert.net>

Evaluation (To be filled by Mentor (Institute))

Criteria	Excellent	Good	Average	Poor	Marks
Work Completion & Implementation (25)	≥90% meaningful work completed	75–89%	50–74%	<50%	/25
Testing, Results & Validation (25)	Strong testing with evidence and analysis	Good testing	Limited testing	No meaningful validation	/25
Technical Understanding (20)	Strong conceptual and technical understanding	Good understanding	Basic understanding	Unable to explain work	/20
Documentation & Progress Presentation (20)	Complete, clear and well documented	Minor gaps	Incomplete documentation	Poor documentation	/20
References & Citations (10)	Relevant, properly cited and formatted	Minor issues	Limited/relevant sources	Missing/improper	/10
TOTAL					/100

Evaluation

Student Signature	Mentor (Company) Signature	Mentor (Institute) Signature



PPSU

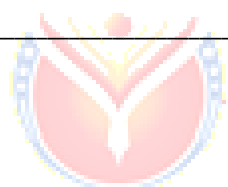
P P SAVANI UNIVERSITY

NAAC
ACCREDITED



School of
Engineering

Mentor (Institute) Remarks



PPSU

P P SAVANI UNIVERSITY

NAAC
ACCREDITED

